

Predicting Resource Necessity from Internal Representations in Large Language Models

Shreya Nupur Shakya | Independent research report | September 2026

Abstract

Large language models can allocate additional inference-time resources through extended reasoning or external tools. We investigate whether pre-generation hidden representations of Qwen3-1.7B predict empirically defined reasoning-compute and tool-access necessity. Labels are derived from repeated evaluations on GSM8K, MATH-500, and When2Tool, and layer-wise linear probes are trained on final prompt-token representations. The best observed five-fold cross-validated AUROCs are 0.881 for reasoning and 0.963 for tool necessity. At selected layers, hidden representations outperform classifiers based on measured input characteristics. Cross-task probes exhibit predictive transfer, but direction-alignment results do not establish a single shared linear necessity direction. Exploratory routing analyses, which reuse some label-construction outcomes, estimate 89.69% success at 31.25% Thinking allocation and 84.57% at 80% tool access. These findings support the feasibility of pre-generation, model-specific resource-necessity prediction under the evaluated conditions; they do not establish validated routing gains or reductions in measured compute or energy.

1. Introduction

Inference strategies for language models can vary in the amount and type of resources used. Adaptive computation has a longer history in neural networks [5], and chain-of-thought prompting has shown benefits for some multi-step reasoning tasks [6]. Qwen3 provides Thinking and NoThinking modes within a single model [1]. Prior work shows that bypassing explicit reasoning can be competitive under particular budgets and tasks [2], while FrugalGPT [3] and RouteLLM [7] study query-dependent model selection. Toolformer studies learning when and how to invoke APIs [8], and When2Tool directly studies pre-generation decoding of tool necessity [4].

We examine two resource decisions within Qwen3-1.7B: whether to enable additional Thinking computation and whether to grant external tool access. Our questions are whether these model-adaptive necessity labels can be decoded before generation, whether their predictability extends beyond measured prompt-level confounds, whether predictive signals transfer between the two decisions, and whether the predictions can support selective routing. We distinguish predictive transfer from evidence for a shared linear representational direction.

2. Related work

Adaptive reasoning and inference. Graves [5] introduced adaptive computation time in recurrent neural networks. Wei et al. [6] demonstrated the benefits of chain-of-thought prompting on multi-step tasks. Qwen3 integrates Thinking and NoThinking modes [1], while Ma et al. [2] study conditions under which NoThinking is effective. FrugalGPT [3] and RouteLLM [7] study query-dependent routing between language models. These works motivate resource allocation as an input-dependent decision, but do not establish the effectiveness of the specific hidden-state policies evaluated here.

Tools and representation probing. Toolformer learns when and how to invoke external APIs [8]. Sun et al. introduce When2Tool, show that tool necessity is linearly decodable from pre-generation representations, and propose Probe&Prefill [4]. Hewitt and Liang [9] caution that probe accuracy alone does not establish how information is represented, motivating control comparisons. Our experiment does not claim novelty in linear probing of tool necessity. Instead, it investigates reasoning-compute necessity alongside tool access, measured-confound controls, cross-task transfer, linear-direction alignment, and resource-budgeted routing in one model.

Benchmarks and interpretation. GSM8K originates from Cobbe et al. [10], and MATH from Hendrycks et al. [11]. The MATH-500 subset is associated with the evaluation in Lightman et al. [12]. Because generated reasoning traces need not faithfully explain model behavior [13], we interpret pre-generation probes as predictive measurements rather than direct evidence of model introspection or causal mechanisms.

3. Methodology

3.1. Behavioral necessity labels

We use Qwen/Qwen3-1.7B (seed 42). Label-generating rollouts use temperature 0.7, top-p 0.8, and four generations per condition. Maximum new-token limits are 512 for NoThinking, 2,048 for Thinking, and 256 for direct tool-answer generation. These are generation limits, not measured token consumption. The reasoning sample contains 100 GSM8K [10]

and 120 MATH-500 [11, 12] problems. For each problem, four stochastic rollouts are evaluated with Thinking and four with NoThinking. Let p_N and p_T denote the corresponding empirical success proportions. A problem is reasoning-necessary if $p_N \leq 0.25$ and $p_T \geq 0.50$, unnecessary if $p_N \geq 0.75$, and ambiguous otherwise. The tool sample contains 180 single-hop When2Tool [4] examples. Four direct-answer rollouts without tools yield p_D . A tool example is necessary if $p_D = 0$, unnecessary if $p_D \geq 0.75$, and ambiguous otherwise. Ambiguous examples are excluded from the primary probe datasets. Tool-necessity labels are operationally defined from direct-answer failure; they do not require showing that tool access improves accuracy on each labeled example. The labels are specific to the evaluated model and generation protocol.

3.2. Pre-generation representations and probes

We apply the Qwen chat template with Thinking disabled and run a single forward pass without generation or tool execution. The final prompt-token representation is extracted from the embedding output and all 28 transformer layers, giving 29 vectors of dimension 2,048 per example. Each representation is evaluated with a separate L2 logistic-regression probe ($C = 1$, balanced class weights). Features are standardized within training folds. Five-fold stratified cross-validation provides out-of-fold predictions, with AUROC as the primary metric and MCC as a secondary metric. The highest observed cross-validated AUROC identifies the layer used in subsequent within-task analyses; this layer-selection procedure is not an independent test-set estimate.

3.3. Confounds and cross-task analyses

We compare confound-only, hidden-only, and combined predictors. Reasoning confounds comprise prompt length, source dataset, and difficulty. Tool confounds comprise prompt length, environment, and difficulty. Categorical confounds are encoded, and selected-layer combined-versus-confound AUROC differences are assessed by paired stratified bootstrap over fixed out-of-fold predictions (5,000 resamples). We evaluate raw cross-task probe transfer in both directions without target-task fitting. An exploratory prompt-length matching analysis is excluded from the reported findings because the saved analysis configuration explicitly invalidates outcome-conditioned matching as a robustness test. We compare linear probe directions using cosine similarity. These analyses address measured characteristics and linear geometry, not all possible confounds or nonlinear structure.

3.4. Resource-budgeted routing

Pre-generation probe scores rank examples for selective allocation of Thinking or tool access. The primary requested budgets are 30% Thinking and 80% tool access. Reasoning routing is compared against Always NoThinking, Always Thinking, and an outcome-informed empirical oracle. Tool routing is compared against Always Direct, Always Tool, and fixed-budget random allocation, with an empirical oracle used diagnostically. The original reasoning routing evaluation reuses the empirical Thinking and NoThinking outcomes used to construct necessity labels. The direct-answer arm of tool routing likewise reuses label-construction outcomes, while its tool-enabled outcomes come from a separate tool-access evaluation. These routing success comparisons are therefore not fully independent of label construction. In addition, reasoning routing uses layer 17, selected upstream using cross-validation across the full retained reasoning sample. Allocation rate measures enabled resource access, not FLOPs, latency, energy, or financial cost.

4. Results

4.1. Dataset composition and layer-wise decoding

After excluding ambiguous examples, the reasoning dataset contains 160 examples (29 necessary, 131 unnecessary), and the tool dataset contains 175 examples (150 necessary, 25 unnecessary). The best observed cross-validated AUROC is 0.8810 at layer 17 for reasoning and 0.9635 at layer 28 for tools (Figure 1). These values describe layer-wise cross-validation within the sampled datasets.

Pre-generation hidden states predict downstream resource necessity

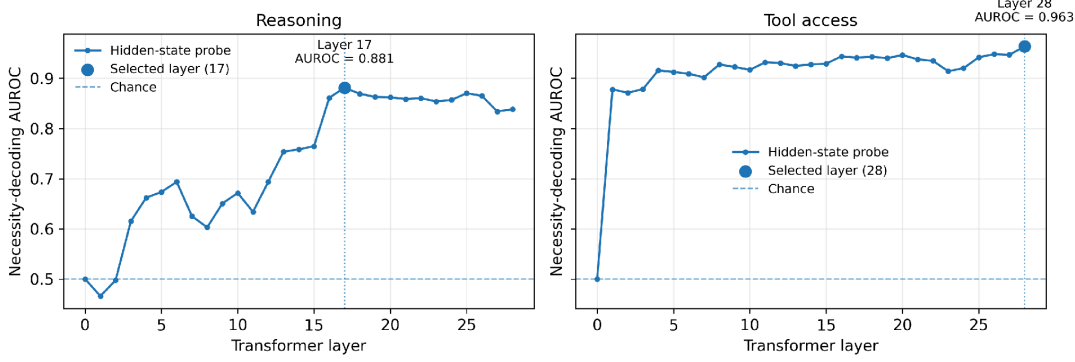


Figure 1. Layer-wise five-fold cross-validated AUROC for reasoning and tool necessity. Original figure from the final experiment artifacts. Layer 0 is at chance because the final prompt-token embedding is constant across examples under the shared chat template; contextual differences emerge only after transformer layers.

4.2. Measured-confound controls

At the selected reasoning layer, confound-only, hidden-only, and combined AUROCs are 0.6881, 0.8810, and 0.8813. The hidden-minus-confound AUROC difference is 0.1929, while the combined-minus-confound difference is 0.1932 (95% paired-bootstrap interval 0.0766–0.3098). At the selected tool layer, the corresponding AUROCs are 0.8981, 0.9635, and 0.9643. The hidden-minus-confound difference is 0.0653, while the combined-minus-confound difference is 0.0661 (95% interval 0.0120–0.1336). For tool necessity, measured confounds are already highly predictive (AUROC 0.8981), and hidden representations provide a smaller improvement over the confound-only baseline than in the reasoning task. Figure 2 displays hidden-minus-confound differences. The confidence intervals refer to the distinct combined-minus-confound comparison. These intervals are conditional on the selected layers and do not account for layer-selection uncertainty.

Hidden-state necessity signals exceed measured prompt confounds

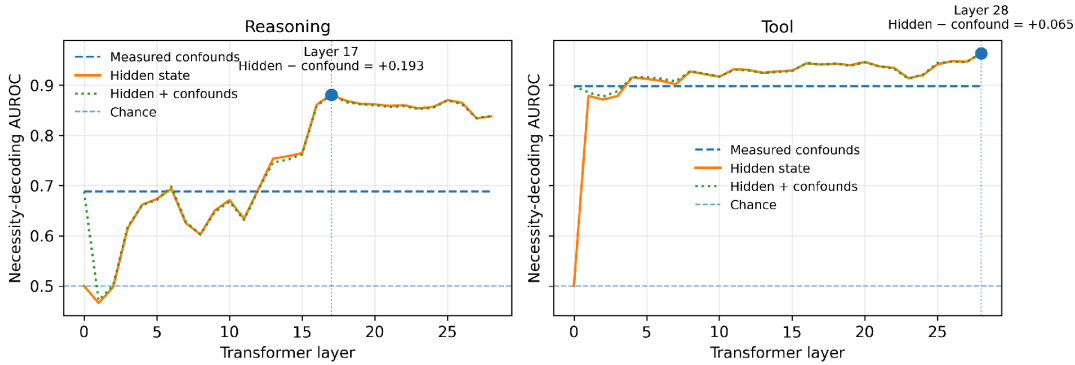


Figure 2. Confound-only, hidden-only, and combined AUROC by layer. Any “Hidden – confound” annotations refer to hidden-only minus confound-only AUROC, not combined-minus-confound. Original experiment figure.

4.3. Cross-task transfer and alignment

Across layers, raw mean transfer AUROC is 0.6246 for Tool → Reasoning and 0.6621 for Reasoning → Tool. The saved analysis configuration invalidates an exploratory prompt-length matching procedure as a robustness test, so matched-transfer results are not reported. Probe directions are nearly orthogonal: mean cosine similarity is 0.0265, with a maximum of approximately 0.12 across 28 comparable layers. The 100-permutation test has insufficient p-value resolution for a Bonferroni threshold of 0.00179 across 28 layers, so a nonsignificant corrected result cannot serve as evidence of absence. Raw cross-task transfer yields above-chance AUROC point estimates, but their statistical reliability and robustness to dataset differences remain unestablished. The small direction similarities do not support a single shared linear necessity direction.

4.4. Selective resource allocation

In the original, non-independent reasoning routing analysis, a requested 30% Thinking budget yields 31.25% allocation and 89.69% estimated success, compared with 80.78% for Always NoThinking and 83.91% for Always Thinking. These rates reuse the same four-rollout outcomes used to define reasoning-necessity labels, so the apparent advantage is not independently validated. At the primary tool operating point, 80% tool access yields 84.57% success, versus 14.14% for Always Direct and 89.29% for Always Tool. The Always Direct rate equals the mean labeling p_{direct} on the 175 retained examples, and the direct arm of selective routing also reuses those labeling outcomes. At this operating point, selective tool routing uses tools less often than Always Tool but has lower estimated success. Figure 3 therefore presents exploratory routing estimates rather than a fully independent evaluation.

Selective routing converts pre-generation signals into resource allocation

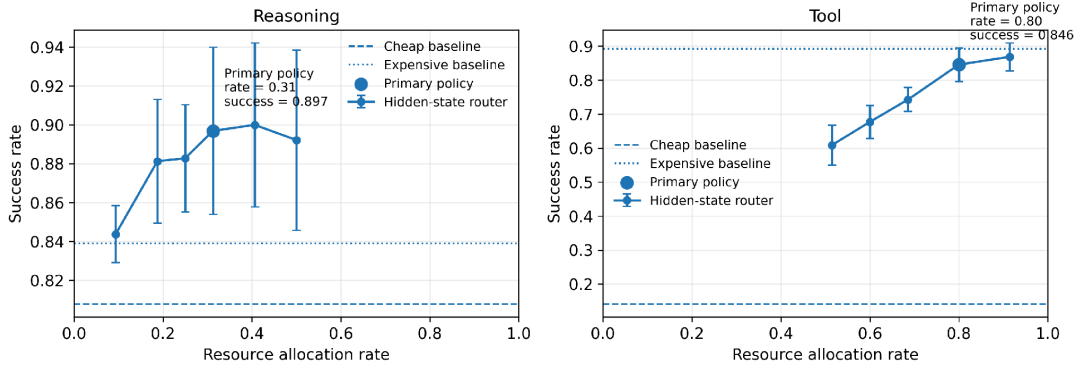


Figure 3. Original routing success versus allocation rate. Both routing curves reuse some label-construction outcomes and are exploratory; neither is an independent test. Original experiment figure.

5. Analysis and limitations

The decoding results establish predictive association between pre-generation representations and model-adaptive necessity labels, not a causal mechanism or model self-awareness. Reasoning routing reuses the empirical success rates used to assign labels, and tool routing reuses the direct-answer outcomes from its label construction. Out-of-fold or nested probe fitting does not remove this outcome reuse. Reasoning routing also inherits layer 17, selected using the full retained reasoning sample before the routing cross-validation. Independent fresh outcomes, with router predictions and operating budgets fixed before evaluation, are needed to validate the routing-performance comparisons. Raw cross-task transfer does not imply common linear geometry; the observed probe-direction cosines are small, while the 100-permutation analysis lacks resolution for the reported multiple-testing threshold.

Generality is limited by the use of one model, three benchmark sources, and small, imbalanced retained datasets. Each empirical success proportion is based on four stochastic rollouts per condition. The reasoning routing evaluation excludes 60 ambiguous examples from the original 220-example sample; these excluded examples have mean NoThinking and Thinking success rates of 13.3% and 17.1%, respectively. A deployed router would not know these labels in advance. Thinking generations were capped at 2,048 new tokens. In the label-construction rollouts, 217 of 880 Thinking generations (24.7%) ended without a closing `</think>` tag, consistent with hitting the token cap; 177 of these were on MATH-500. These runs were correct 4.6% of the time, compared with 85.7% for generations containing a closing `</think>` tag. Fresh-rollout evaluation should therefore record and control for truncation. Layer selection based on the same cross-validation results can favor optimistic peak-layer estimates. The controlled analyses consider measured prompt characteristics, while the geometric analysis tests linear directions only. Neither token consumption nor actual latency, FLOPs, energy, or monetary cost savings are established by the allocation-rate results.

6. Conclusion

Pre-generation hidden representations of Qwen3-1.7B contain linearly decodable information predictive of empirically defined reasoning and tool necessity. At the best observed cross-validated layers, this predictive information extends beyond measured prompt-level confounds; the peak AUROCs are not independent held-out test estimates. Raw cross-task transfer yields above-chance point estimates without established statistical reliability; the small cosine similarities between probe directions do not support a single shared linear necessity direction. Both reported routing analyses reuse some

label-construction outcomes and are exploratory rather than independently validated. Fresh-rollout evaluation that records truncation, replication across models and tasks, and direct resource-cost measurement remain necessary.

References

- [1] Yang, A., et al. (2025). Qwen3 Technical Report. arXiv:2505.09388. <https://arxiv.org/abs/2505.09388>
- [2] Ma, W., He, J., Snell, C., Griggs, T., Min, S., & Zaharia, M. (2025). Reasoning Models Can Be Effective Without Thinking. arXiv:2504.09858. <https://arxiv.org/abs/2504.09858>
- [3] Chen, L., Zaharia, M., & Zou, J. (2024). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. Transactions on Machine Learning Research. <https://mlanthology.org/tmlr/2024/chen2024tmlr-frugalgpt/>
- [4] Sun, C.-E., Liu, L., Yan, G., Wang, Z., & Weng, T.-W. (2026). LLM Agents Already Know When to Call Tools—Even Without Reasoning. arXiv:2605.09252. <https://arxiv.org/abs/2605.09252>
- [5] Graves, A. (2016). Adaptive Computation Time for Recurrent Neural Networks. arXiv:1603.08983. <https://arxiv.org/abs/1603.08983>
- [6] Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903. <https://arxiv.org/abs/2201.11903>
- [7] Ong, I., et al. (2025). RouteLLM: Learning to Route LLMs from Preference Data. International Conference on Learning Representations. <https://mlanthology.org/iclr/2025/ong2025iclr-routellm/>
- [8] Schick, T., et al. (2023). Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html
- [9] Hewitt, J., & Liang, P. (2019). Designing and Interpreting Probes with Control Tasks. EMNLP-IJCNLP, 2733–2743. <https://aclanthology.org/D19-1275/>
- [10] Cobbe, K., et al. (2021). Training Verifiers to Solve Math Word Problems. arXiv:2110.14168. <https://arxiv.org/abs/2110.14168>
- [11] Hendrycks, D., et al. (2021). Measuring Mathematical Problem Solving With the MATH Dataset. Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1 (NeurIPS 2021). https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html
- [12] Lightman, H., et al. (2024). Let’s Verify Step by Step. International Conference on Learning Representations (ICLR 2024). <https://mlanthology.org/iclr/2024/lightman2024iclr-let/>
- [13] Turpin, M., Michael, J., Perez, E., & Bowman, S. R. (2023). Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting. Advances in Neural Information Processing Systems 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/ed3fea9033a80fea1376299fa7863f4a-Abstract-Conference.html